Name:Sameer Gupta

Roll No:252010036

Cafe Sales Data Cleaning and Analysis

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load data
df = pd.read_csv('dirty_cafe_sales.csv')

# Standardize missing values
df.replace(['UNKNOWN', 'ERROR', ' '], np.nan, inplace=True)
df['Total Spent'] = pd.to_numeric(df['Total Spent'], errors='coerce')
df['Price Per Unit'] = pd.to_numeric(df['Price Per Unit'], errors='coerce')
df['Quantity'] = pd.to_numeric(df['Quantity'], errors='coerce')
```

1. Count the number of null entries for every attribute

In [2]:
```
null_counts = df.isnull().sum()
print("Null entries per attribute:\n", null_counts)
```

```
Null entries per attribute:
 Transaction ID         0
Item                 969
Quantity             479
Price Per Unit       533
Total Spent          502
Payment Method      3178
Location            3961
Transaction Date     460
dtype: int64
```

2. Fill null entries with appropriate decisions

In [3]:
```
# a) if quantity is blank then fill it using 'total spent' and 'cost per unit'
df['Quantity'] = df['Quantity'].fillna(df['Total Spent'] / df['Price Per Unit'])

# b) if item is blank then fill it using cost per unit
# We define a mapping for common prices to items based on existing frequencies
price_to_item = {2.0: 'Coffee', 3.0: 'Cake', 1.0: 'Cookie', 5.0: 'Salad', 4.0: 'San
df['Item'] = df['Item'].fillna(df['Price Per Unit'].map(price_to_item))

# c) if the transaction date is blank then see if this can be filled by multi colum
df = df.sort_values(by='Transaction ID')
```

```
# UPDATED LINE TO FIX WARNING:
df['Transaction Date'] = df['Transaction Date'].ffill().bfill()

# d) try to fill payment mode
mode_payment = df['Payment Method'].mode()[0]
df['Payment Method'] = df['Payment Method'].fillna(mode_payment)

print("Remaining nulls:\n", df.isnull().sum())
```

```
Remaining nulls:
 Transaction ID         0
Item                  54
Quantity              38
Price Per Unit       533
Total Spent          502
Payment Method         0
Location            3961
Transaction Date       0
dtype: int64
```

3. Split the transaction date to 3 columns as date, month and year

In [4]:
```
df['Transaction Date'] = pd.to_datetime(df['Transaction Date'])
df['Day'] = df['Transaction Date'].dt.day
df['Month'] = df['Transaction Date'].dt.month
df['Year'] = df['Transaction Date'].dt.year
df.head()
```

Out[4]:

	Transaction ID	Item	Quantity	Price Per Unit	Total Spent	Payment Method	Location	Transaction Date	Da
2987	TXN_1000555	Tea	1.0	1.5	1.5	Credit Card	In-store	2023-10-19	1
115	TXN_1001832	Salad	2.0	5.0	10.0	Cash	Takeaway	2023-10-19	1
418	TXN_1002457	Cookie	5.0	1.0	5.0	Digital Wallet	Takeaway	2023-09-29	2
3417	TXN_1003246	Juice	2.0	3.0	6.0	Digital Wallet	NaN	2023-02-15	1
4366	TXN_1004184	Smoothie	1.0	4.0	4.0	Credit Card	In-store	2023-05-18	1

4. Aggregate sales based on day, month and year

In [5]:
```
daily_sales = df.groupby(['Year', 'Month', 'Day'])['Total Spent'].sum().reset_index
print("Daily Sales Aggregation:\n", daily_sales.head())
```

```
Daily Sales Aggregation:
    Year  Month  Day  Total Spent
0  2023      1    1        176.5
1  2023      1    2        171.5
2  2023      1    3        148.0
3  2023      1    4        289.5
4  2023      1    5        368.0
```

5. Aggregate the query 4 based on products

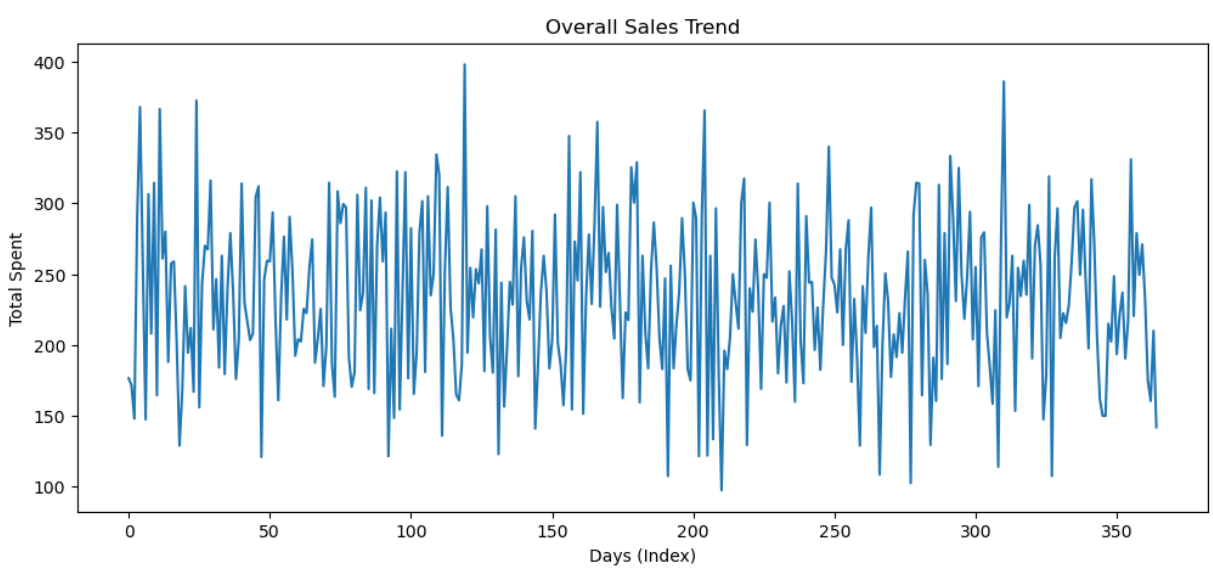
In [6]:
```
product_sales = df.groupby(['Item', 'Year', 'Month', 'Day'])['Total Spent'].sum().r
print("Product Daily Sales Aggregation:\n", product_sales.head())
```

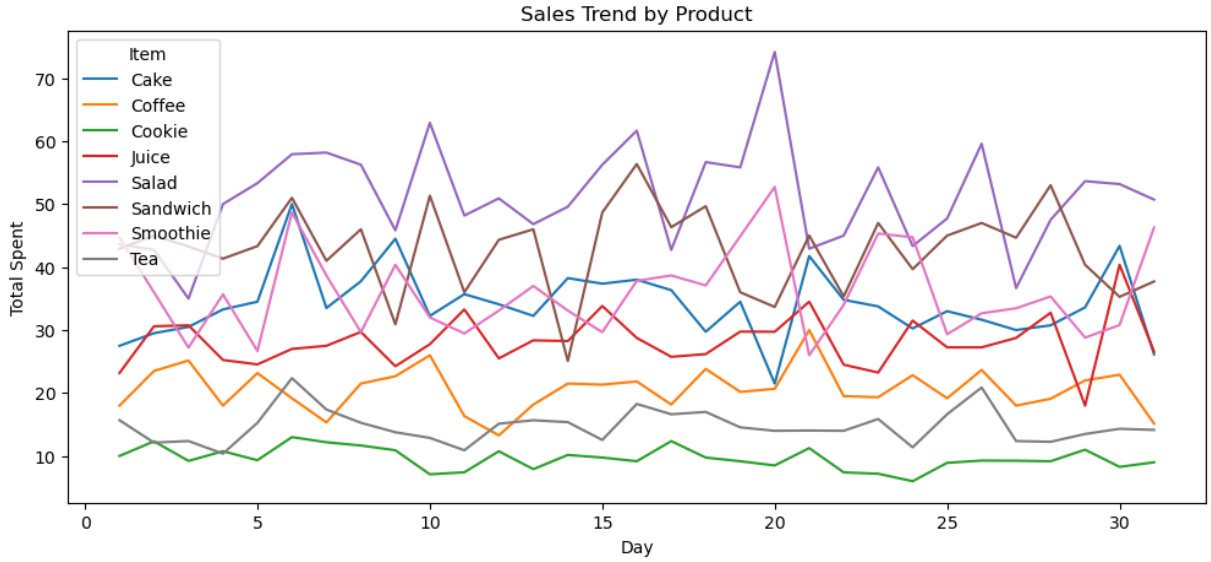
```
Product Daily Sales Aggregation:
    Item  Year  Month  Day  Total Spent
0  Cake  2023      1    1         42.0
1  Cake  2023      1    2         21.0
2  Cake  2023      1    3         39.0
3  Cake  2023      1    4         21.0
4  Cake  2023      1    5         30.0
```

6. Visualize the query 4 and 5

In [7]:
```
plt.figure(figsize=(12, 5))
plt.plot(daily_sales.index, daily_sales['Total Spent'])
plt.title('Overall Sales Trend')
plt.xlabel('Days (Index)')
plt.ylabel('Total Spent')
plt.show()

plt.figure(figsize=(12, 5))
# UPDATED LINE TO FIX WARNING: replaced ci=None with errorbar=None
sns.lineplot(data=product_sales, x='Day', y='Total Spent', hue='Item', errorbar=Non
plt.title('Sales Trend by Product')
plt.show()
```





7. Create a dashboard for the same

In [8]:
```
import ipywidgets as widgets
from IPython.display import display

def plot_dashboard(item_filter):
    plt.figure(figsize=(10, 4))
    if item_filter == 'All':
        data = daily_sales
        plt.plot(data.index, data['Total Spent'])
    else:
        data = product_sales[product_sales['Item'] == item_filter]
        plt.plot(data['Day'], data['Total Spent'])
    plt.title(f'Sales for {item_filter}')
    plt.ylabel('Total Spent')
    plt.show()

items = ['All'] + list(df['Item'].dropna().unique())
dropdown = widgets.Dropdown(options=items, description='Select Item:')
widgets.interactive(plot_dashboard, item_filter=dropdown)
```

Out[8]:
```
interactive(children=(Dropdown(description='Select Item:', options=('All', 'Tea',
'Salad', 'Cookie', 'Juice', …
```